

自动化平台

一、平台使用框架介绍

前端框架: vue + elementui + axios

后端框架: django + restframework + apscheduler + requests + telnetLib

中间件: redis + mysql

二、功能介绍使用

1、项目列表 --可以理解为服务模块

功能: 添加、删除、编辑、禁用



编辑

* 项目名称 测试项目

* 类型 Web

* 版本号 1.0

描述 1111

取消 提交

2、Host 配置 --主要功能是域名切换

功能: 添加、删除、编辑、禁用

Http 类型可以输入 ip 加端口或域名

Dubbo 服务只支持 IP 加端口

☐ 名称	HOST	描述
☐ 测试项目	m.shanzhen.me	善诊
☐ Dubbo项目	47.114.94.186:20890	Dubbo项目接口

3、API 接口

功能：查询、添加、删除、编辑、分组、下载接口文档、导入接口、校验等
添加接口

《 接口列表

保存取消

* 接口分组: 查询接口

* 接口名称: 名称

状态: 启用

URL: POST HTTP

请求协议

接口URL地址

请求方式

header 请求头

请求参数

请求参数

DUBBO 接口录入

* 接口名称: 排期必传字段为空

URL: DUBBO DUBBO

dubbo方法路径

请求头部

请求类路径

断言设置

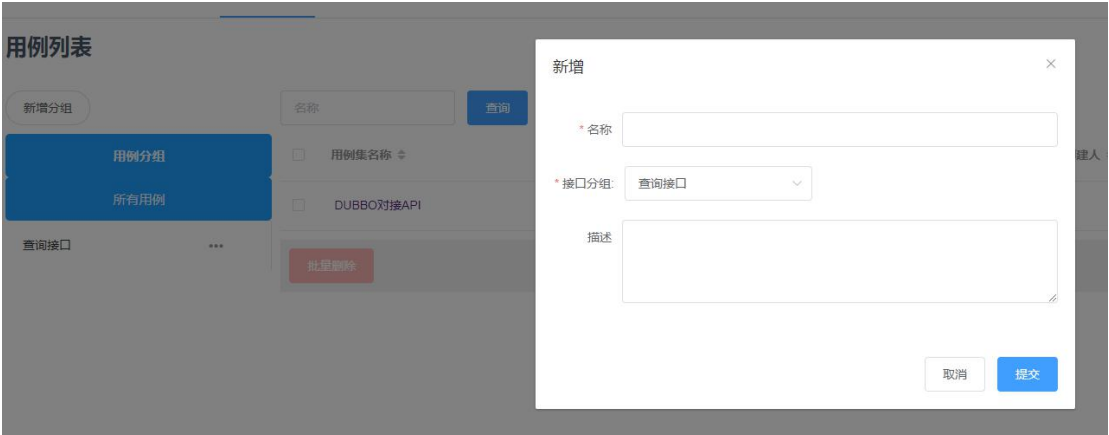
测试结果校验

不校验 校验http状态 JSON校验 完全校验 正则校验 respMsg

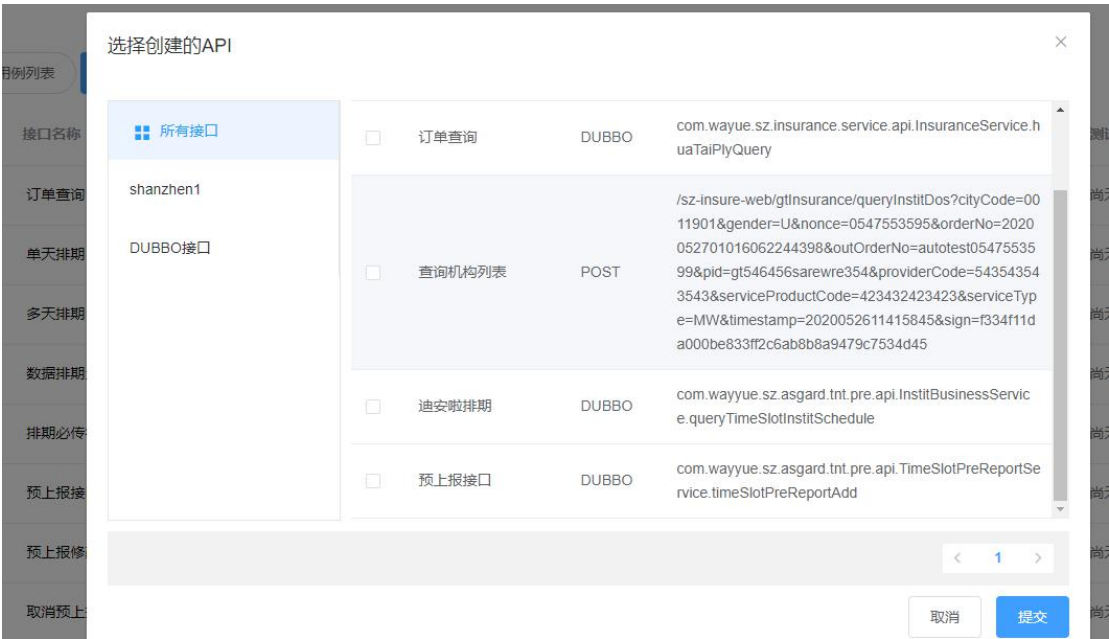
200

成功

自动化测试
新增用例集合



可以从原有的 api 接口里面拉数据



手动跑测试用例

用例列表

+ 已有接口

+ 新建接口

测试全部

环境必选

测试环境

#	接口名称	接口地址	测试结果	操作
1	订单查询1	DUBBO com.wayue.sz.insurance.service.api.InsuranceService.huaTaiPlyQuery	尚无测试结果	测试 修改 删除
2	单天排期	DUBBO com.wayyue.sz.asgard.tnt.pre.api.InstitBusinessService.queryTimeSlotInstitSchedule	尚无测试结果	测试 修改 删除
3	多天排期	DUBBO com.wayyue.sz.asgard.tnt.pre.api.InstitBusinessService.queryTimeSlotInstitSchedule	尚无测试结果	测试 修改 删除
4	数据排期为空	DUBBO com.wayyue.sz.asgard.tnt.pre.api.InstitBusinessService.queryTimeSlotInstitSchedule	尚无测试结果	测试 修改 删除
5	排期必传字段为空	DUBBO com.wayyue.sz.asgard.tnt.pre.api.InstitBusinessService.queryTimeSlotInstitSchedule	尚无测试结果	测试 修改 删除
6	预上报接口	DUBBO com.wayyue.sz.asgard.tnt.pre.api.TimeSlotPreReportService.timeSlotPreReportAdd	尚无测试结果	测试 修改 删除
7	预上报修改	DUBBO com.wayyue.sz.asgard.tnt.pre.api.TimeSlotPreReportService.timeSlotPreReportAdd	尚无测试结果	测试 修改 删除

定时任务配置

定时任务

×

* 任务名称：

撒打算的

* 类型：

循环

* 间隔：

间隔

执行时间间隔

请选择

* 执行时间：

🕒

开始日期

至

结束日期

* HOST：

测试环境

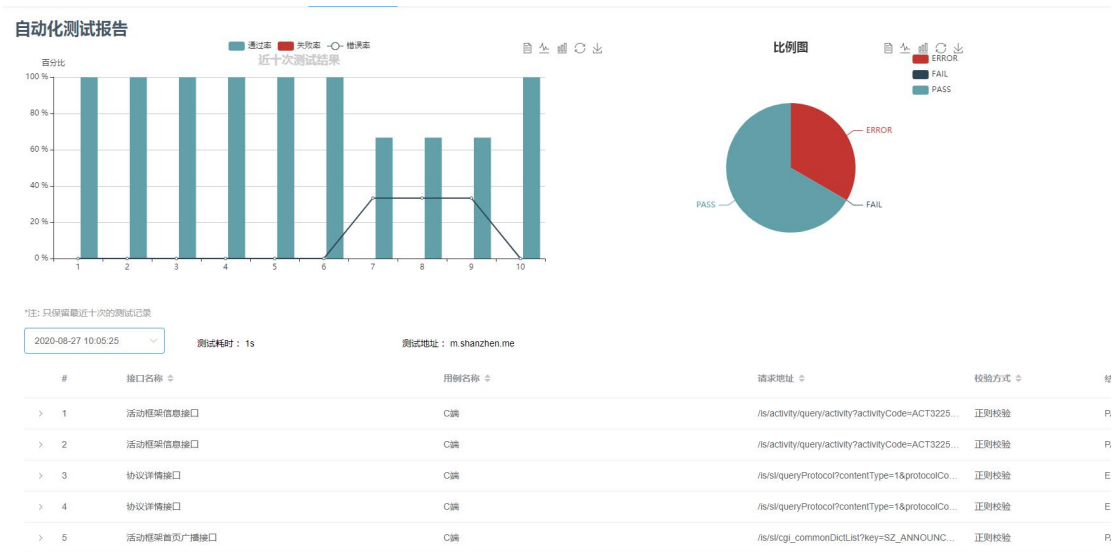
选择用例：

选择测试用例

选择需要定时跑的用例集合

创建

4、自动化报告



5、定时任务

任务管理:

名称

当前

新增任务

#	任务名称	任务类型	执行频率	时分钟	开始时间	结束时间	定时执行任务记录
1	测试循环20	循环	20	分	2020-08-24 17:45:25	2020-08-31 17:43:25	查看执行记录 删除
2	测试循环10	循环	10	分	2020-08-24 17:45:25	2020-08-31 17:43:25	查看执行记录 删除
3	测试循环1000	循环	5	分	2020-08-24 17:45:25	2020-08-31 17:43:25	查看执行记录 删除

<

1

定时任务记录

新增分组

用例分组

所有用例

测试case

...

#	任务名称	任务是否成功	执行任务id	任务持续时间	执行时间	所属项目
1	测试循环20	失败	1623	0	2020-08-27 10:05:25	测试-test
2	测试循环20	成功	1617	0	2020-08-27 09:45:25	测试-test
3	测试循环20	成功	1610	0	2020-08-27 09:25:25	测试-test
4	测试循环20	成功	1604	0	2020-08-27 09:05:25	测试-test
5	测试循环20	成功	1596	0	2020-08-27 08:45:25	测试-test
6	测试循环20	成功	1588	0	2020-08-27 08:25:25	测试-test
7	测试循环20	成功	1582	0	2020-08-27 08:05:25	测试-test
8	测试循环20	成功	1574	0	2020-08-27 07:45:25	测试-test
9	测试循环20	成功	1569	0	2020-08-27 07:25:25	测试-test
10	测试循环20	成功	1561	0	2020-08-27 07:05:25	测试-test

<

1

2

3

4

5

...

20

>

6、全局变量

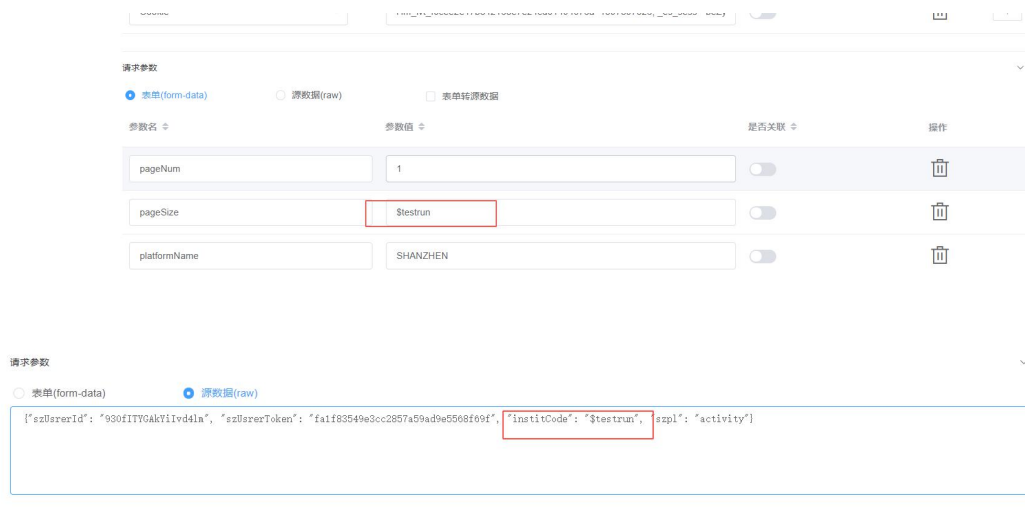
新增变量：



编辑保存



引用变量



三、前后端部署

1、前端部署

/opt/test-plaform/inferfince-robot/frontend 目录下 npm install

执行 npm run build 打包成 dist 文件

NGINX 配置之后，执行/opt/tengine/sbin/nginx -s reload

```

#gzip on;

server {
    listen 8888;
    server_name 172.16.21.241;
    #charset koi8-r;
    #access_log logs/host.access.log main;

    location / {
        #proxy_pass http://172.16.21.241:8888/;
        root /opt/test-plaform/inferfince-robot/frontend/dist/;
        index index.html index.htm;
        try_files $uri $uri/ @router;
    }

    #error_page 404 /404.html;

    # redirect server error pages to the static page /50x.html
    #
    error_page 500 502 503 504 /50x.html;
    location = /50x.html {
        root html;
    }

    # proxy the PHP scripts to Apache listening on 127.0.0.1:80
    #
    #location ~ \.php$ {
    #    proxy_pass http://127.0.0.1;
    #}

    # pass the PHP scripts to FastCGI server listening on 127.0.0.1:9000
    #
    #location ~ \.php$ {
    #    root html;
    #    fastcgi_pass 127.0.0.1:9000;
    #    fastcgi_index index.php;
    #    fastcgi_param SCRIPT_FILENAME /scripts$fastcgi_script_name;
    #    include fastcgi_params;
    #}

    # deny access to .htaccess files, if Apache's document root
    # concurs with nginx's one
    #

```

IP加端口号

配置dist地址

2、后端部署

/opt/test-plaform/inferfince-robot/script 执行 /opt/python3/bin/uwsgi --ini uwsgi.ini
启动就好了

```
[uwsgi]
# 项目目录
chdir=/opt/Jacoco/rateCoverPlaform
# 指定项目的application
module=rateCoverPlaform.wsgi:application
# 指定sock的文件路径
socket=/opt/Jacoco/rateCoverPlaform/script/uwsgi.sock
# 进程个数
workers=1
pidfile=/opt/Jacoco/rateCoverPlaform/script/uwsgi.pid
# 指定IP端口 云服务器需要改成 0.0.0.0
http=0.0.0.0:19000
#指定静态文件
static-map=/static=/opt/Jacoco/rateCoverPlaform/static
# 启用主进程
master=true
# 自动移除unix Socket和pid文件当服务停止的时候
vacuum=true
# 序列化接受的内容, 如果可能的话
thunder-lock=true
# 启用线程
enable-threads=true
# 设置自中断时间
harakiri=30
# 设置缓冲
post-buffering=4096
# 设置日志目录
daemonize=/opt/Jacoco/rateCoverPlaform/script/uwsgi.log
```

停止服务 /opt/python3/bin/uwsgi --stop uwsgi.pid